ITDS Project Plan

Name: Laxmidhar Panda
Roll No: 24110185
Course: CS328 Introduction to Data Science
Date: 12-10-2026

1. Motivation

Retrieving similar graphs from a massive corpus is a foundational problem in chemistry, biology, and cybersecurity. A naive approach comparing a query graph to all items in a corpus is completely unscalable because exact graph edit distance and subgraph isomorphism checks are computationally prohibitive. While traditional chemoinformatics relies on discrete, set-based fingerprinting (reducing molecules to sets of structural shingles), modern representation learning allows us to embed these graphs into continuous geometric spaces. This project explores how to build a scalable graph retrieval algorithm, directly answering the course prompt on designing graph embeddings combined with locality-sensitive indexing. By shifting from discrete set overlaps to learned geometric spaces, we can dramatically improve both the quality and speed of retrieving molecular structural analogs.

2. Objectives and Hypotheses

This project tests two primary hypotheses regarding graph retrieval pipelines:

- **H1 (Representation Quality):** A learned message-passing model (Graph Isomorphism Network) trained via Supervised Contrastive Learning will significantly outperform discrete, set-based baseline algorithms (Morgan fingerprints + MinHash) in ranking quality metrics such as MAP@10 and Recall@10.
- **H2 (Search Scalability):** When serving the learned continuous vectors, a Hierarchical Navigable Small World (HNSW) proximity graph will dominate random Hyperplane LSH in query latency, proving that the indexing mechanism is just as critical as the embedding quality for sublinear retrieval.

3. Background

Our retrieval pipeline maps to two distinct mathematical approaches covered in the course:

- **Set-Based Hashing (Baseline):** Molecules can be represented as sets of structural shingles. To avoid $O(N^2)$ comparisons, we can estimate Jaccard similarity using MinHash permutations, where $\Pr[h_{\min}(A) = h_{\min}(B)] = J(A, B)$. Banding these signatures generates a steep candidate-selection S-curve.
- **Vector-Based Indexing (Upgrade):** Instead of discrete sets, we can embed graphs as continuous vectors in $\mathbb{R}^d$. Searching these vectors exactly requires an exhaustive linear scan, which is too slow for production. We can accelerate this using Hyperplane LSH (where collision probability translates to geometric angle) or FAISS HNSW, which navigates a multi-scale proximity graph.

4. Methodology

Data.

I will use the ZINC dataset (a standardized subset of roughly 12,000 molecular graphs) available via PyTorch Geometric. Nodes represent atoms with discrete features (atomic number, charge), and edges represent chemical bonds. The graphs are small enough to keep experimentation rapid while allowing real-world retrieval testing.

Implementation.

- **Set-Based Baseline:** Use `RDKit` to extract Extended-Connectivity Fingerprints (ECFP). I will write a custom MinHash implementation with k independent hash permutations and apply a banding structure (b bands, r rows) to surface candidate near-duplicates.
- **Neural Representation:** Build a 4-layer Graph Isomorphism Network (GIN) encoder with global sum and mean pooling to map each graph to a 256-dimensional L_2-normalized vector. The network will be trained using a Supervised Contrastive (SupCon) loss based on structural similarity classes to explicitly shape the embedding geometry.
- **Indexing & Serving:** Index the learned continuous embeddings using both a custom Hyperplane LSH (SimHash) implementation and the `FAISS` library's HNSW algorithm ($M = 32$, $efConstruction = 200$).

Experiments.

- **The 4-Way Ablation:** Compare the retrieval quality and latency across four combinations: 1) ECFP + MinHash, 2) GIN + Hyperplane LSH, 3) ECFP + HNSW, and 4) GIN + HNSW.(This is my current estimate of architecture I would use, I will try to improve and add more ablation in this too.)
- **Evaluation Metrics:** Measure MAP@10, Recall@10, and Mean Reciprocal Rank (MRR) to quantify embedding quality, alongside mean single-query latency (in milliseconds) to quantify search speed.
- **S-Curve Verification:** Plot the empirical candidate probability against true Jaccard similarity for the baseline MinHash pipeline to visualize the phase transition threshold.

5. Deliverables

- A concise written report comparing the effectiveness of simple set-based sketches versus learned neural embeddings on specific graph queries.
- A Pareto-frontier plot showcasing the Quality-Latency trade-off (MAP@10 vs. Latency) for the four tested architectures.
- A clean, documented Python codebase containing the custom MinHash, GIN encoder, and evaluation scripts.

6. Effort and Schedule

This project is scoped for approximately 15–20 hours of individual work over four weeks and will be finished by mid November.

- **Week 1 (Baselines):** Ingest ZINC dataset. Implement ECFP fingerprint extraction and the MinHash + Banding algorithm.
- **Week 2 (Neural Encoder):** Build the GIN architecture in PyTorch Geometric and implement the SupCon training loop.
- **Week 3 (Sublinear Indexing):** Extract the 256-D corpus embeddings and construct both the Hyperplane LSH and FAISS HNSW search indices.
- **Week 4 (Benchmarking):** Run the 4-way ablation experiments, measure MAP@10 and latency, generate plots, and write the final report.